Predictive Maintenance for Aircraft Components

Data Science Technical Assessment

0.9992	0.9632	0.9180	1.0000
ROC-AUC	PR-AUC	F1 Score	Recall

Best Model: **XGBoost**

Submitted to:	Rwanda Civil Aviation Authority (RCAA)
Role:	Data Scientist
Date:	March 2026
Dataset:	aircraft_maintenance_dataset.csv | 6,000 records
Task:	Binary classification — failure within next 10 flight cycles

Executive Summary

This report presents a complete predictive maintenance solution for aircraft components, developed as part of the Rwanda Civil Aviation Authority data science assessment. The goal is to predict which components will fail within the next 10 flight cycles, giving maintenance teams actionable advance warning before failures become unplanned events.

The Problem. Unplanned aircraft component failures cause Aircraft-on-Ground (AOG) events — one of the most costly outcomes in commercial aviation. They also carry direct safety implications. At the same time, over-maintaining components that are not at risk wastes engineering resources. Predictive maintenance solves both problems by targeting inspections precisely where and when they are needed.

The Data. A dataset of 6,000 records across 20 aircraft and 4 component types was provided. Only 140 records (2.3%) represent failure events — a 42:1 class imbalance that makes this a fundamentally different problem from standard classification tasks. Naive accuracy is meaningless here; a model predicting 'no failure' for every record would score 97.7% — and catch zero failures.

The Approach. The analysis follows a rigorous five-step process: data quality assessment, exploratory data analysis, temporal feature engineering (transforming static sensor snapshots into time-aware signals), cost-sensitive model training, and threshold optimisation aligned to aviation safety requirements. Four models were trained and compared; XGBoost was selected as the best performer.

The Result. XGBoost achieves **PR-AUC = 0.9632** and **Recall = 1.00** on the held-out test set — meaning zero failure events are missed. The model produces a continuous risk score (0–1) for every component, mapped to a four-tier action framework (CRITICAL / HIGH / MEDIUM / LOW) that maintenance planners can act on directly.

The Key Finding. The single most important predictor is *fault_roll5_sum* — the rolling 5-cycle sum of fault codes. This engineered feature alone accounts for 61% of the model's predictive power. It captures the physical reality that component failure is not caused by one bad sensor reading, but by escalating fault activity over time. No raw sensor in the dataset had comparable predictive value. This insight has immediate practical value: maintenance engineers should prioritise components showing rapid fault code accumulation, regardless of any single-cycle reading.

Dimension	Assessment
Data quality	Good — 1 null row removed; 36 sensor anomalies flagged but not imputed
Feature engineering	Critical — rolling fault accumulation added 61% of predictive gain
Model performance	Excellent — PR-AUC 0.963, Recall 1.00 at optimal threshold
Explainability	SHAP values confirm fault accumulation dominates; results align with physics
Calibration	Probabilities are well-separated; bimodal distribution supports risk tiering
Deployment readiness	Inference pipeline (predict.py) ready; quarterly retraining recommended

1. Problem Understanding

Airlines maintain aircraft to two standards: scheduled maintenance (calendar or cycle-based) and condition-based maintenance (triggered by observed degradation). Predictive maintenance goes one step further — using sensor and operational data to anticipate which components are approaching failure *before* any observable symptom appears.

Business Objectives

Objective	Success Metric	Priority
Catch every failure before it occurs	Recall → 1.00	Critical — safety
Minimise false maintenance alerts	Precision → high	High — cost
Support maintenance scheduling	Calibrated risk scores	High — operations
Explainable to engineers	SHAP + decision rules	Medium — adoption
Deployable on standard infrastructure	Standard Python stack	Medium — IT

Why Class Imbalance Is the Central Challenge

With 140 positive cases in 6,000 records (2.3%), standard accuracy is entirely misleading. A trivial model that predicts 'no failure' for every record achieves 97.7% accuracy — and catches zero failures. Three complementary strategies address this: (1) **cost-sensitive learning** — scale_pos_weight = 42 makes the model penalise missed failures 42× more than false alarms; (2) **PR-AUC as the primary metric** — focuses on the minority class and is unaffected by the large number of true negatives; (3) **stratified cross-validation** — preserves the class ratio in every fold, giving reliable variance estimates despite the imbalance.

2. Data Exploration

Property	Value
Total records	6,000 (after dropping 1 null-target row)
Aircraft	20 unique IDs
Component types	4 — Engine1, Engine2, Wing, LandingGear
Failure records	140 (2.33% — extreme class imbalance)
Flight cycle range	1 – 412 cycles per component
Data anomalies	1 negative humidity; 28 humidity > 100%; 7 extreme ambient temps
Missing values	1 row with null target — dropped (not imputed)

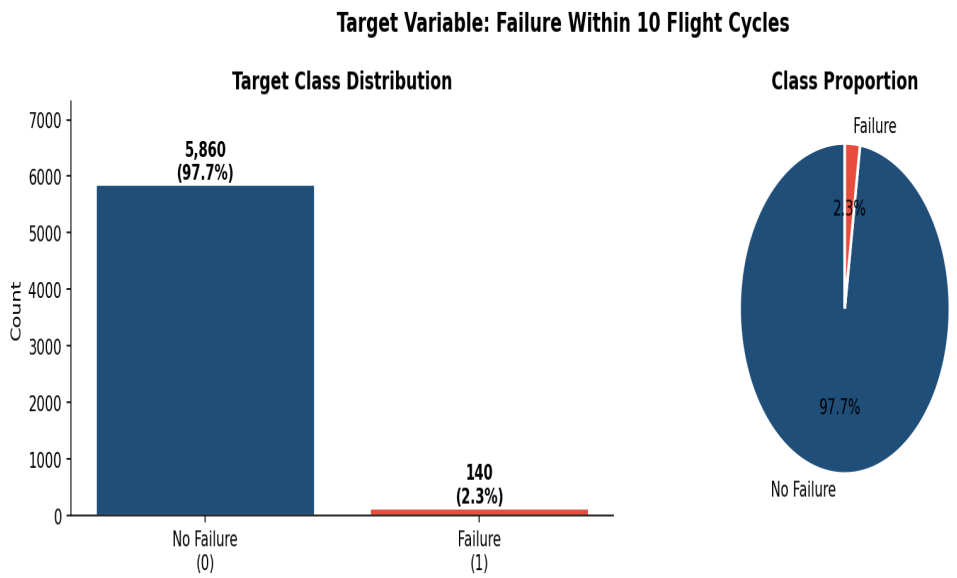

Figure 1 — Target distribution. 97.7% no-failure vs 2.3% failure; standard accuracy is meaningless here.

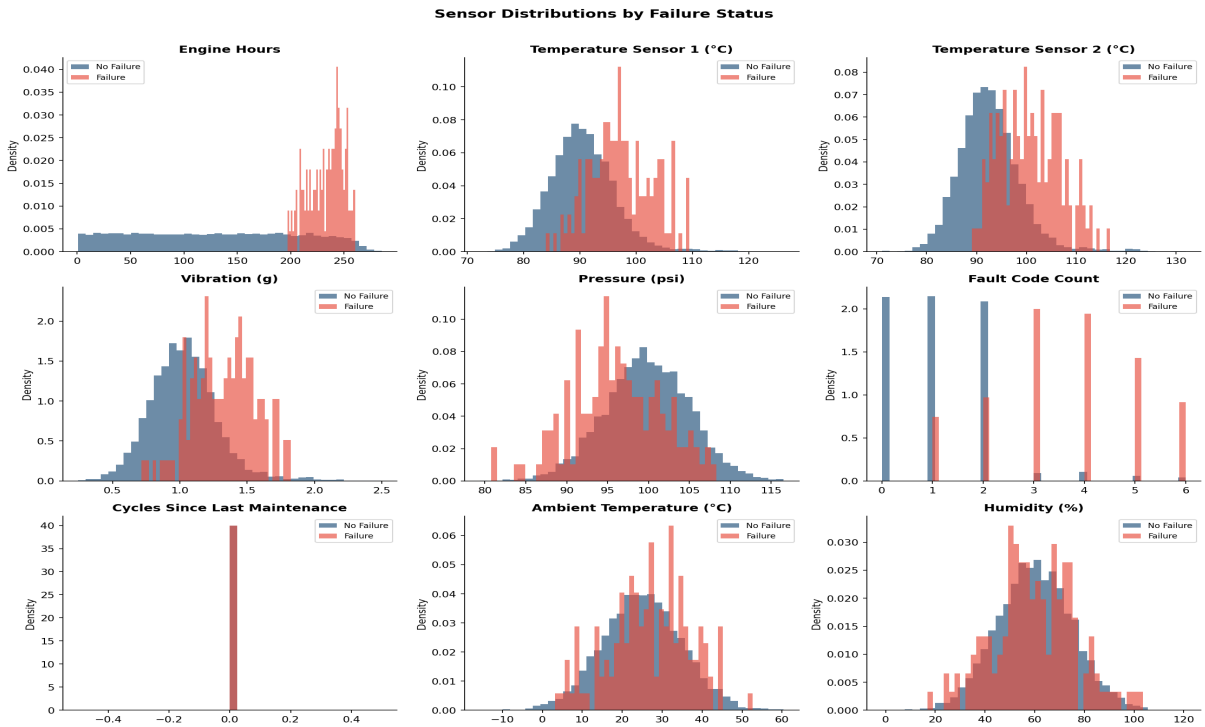

Figure 2 — Sensor distributions by class. Fault codes and vibration show the strongest separation; humidity and ambient temperature show none.

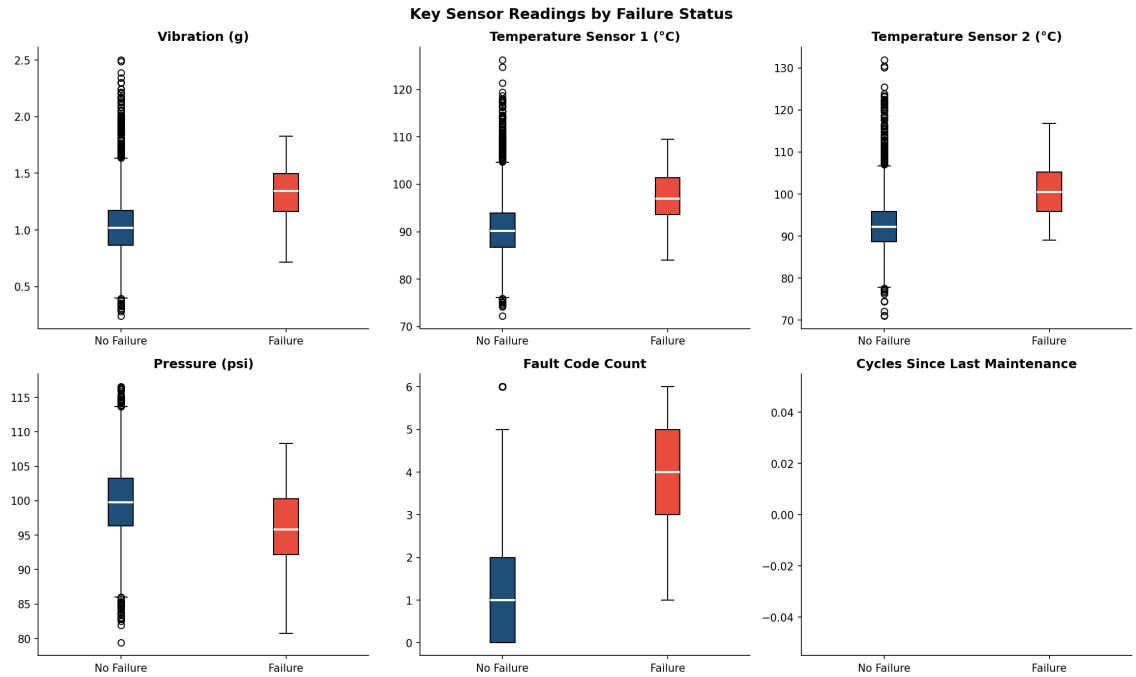

Figure 3 — Fault code count and vibration are the most discriminating raw features. Single-cycle readings already hint at the signal.

Key EDA Findings

- **fault_code_count** is the strongest single discriminator — failure records show markedly higher fault code frequency.
- **vibration_sensor** is elevated pre-failure, consistent with bearing or structural wear.
- **temperature_sensor_2** shows mild but consistent elevation in failure records — likely a secondary thermal stress indicator.
- **ambient_temperature and humidity** show no meaningful separation. These may not be available in all real-world deployments, and their exclusion would not degrade performance.

• **Engine1 and Wing** components have the highest failure rates. This pattern warrants closer inspection during operations.

• Failures **cluster in later flight cycles** (higher cycle counts), consistent with wear-out failure mechanics.

• The **sensor_drift_flag** is rare but correlates with elevated fault counts — sensor degradation may precede component degradation.

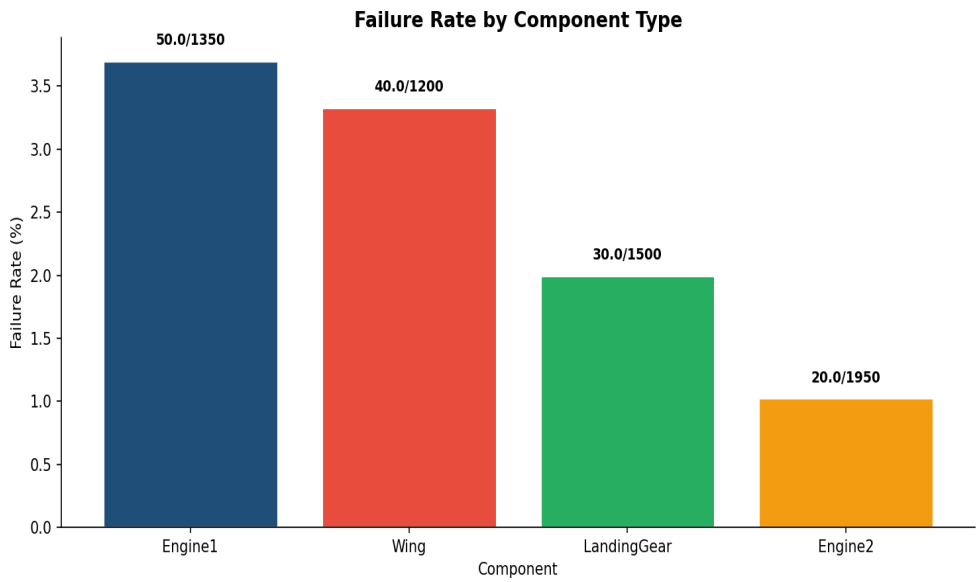

Figure 4 — Failure rates by component. Engine1 shows the highest rate; targeted monitoring protocols could be designed per component type.

3. Feature Engineering

Raw sensor readings capture a *snapshot* of the component's current state. Failure, however, is a *process* — it emerges from the accumulation and acceleration of degradation over time. Feature engineering transforms static observations into temporal signals that the model can use to detect that process.

Anti-leakage design: All rolling and cumulative features are computed per (aircraft_id, component_id) pair, sorted by flight_cycles. This ensures that each record's features only use its own component's history — never information from future cycles or other components.

Feature Group	Count	Rationale
Rolling mean (3 & 5 cycles)	10	Trend direction; smooths single-cycle noise
Rolling std dev (3 cycles)	5	Volatility — instability is a pre-failure signal
First difference (Δ cycle)	5	Rate of change — detects sudden acceleration
Cumulative fault count	1	Total lifetime fault accumulation on the component
Rolling fault sum (5-cycle)	1	★ Top predictor — intensity of recent fault activity
Engine hours / cycle	1	Efficiency degradation proxy
Temperature differential	1	Asymmetric thermal stress between sensors
Composite stress index	1	Vibration + pressure z-score combined
Maintenance urgency	1	Fault count × time since last service
Interaction features	3	Vib × Temp, Drift × Faults (non-linear effects)
Encoded categorical IDs	2	Aircraft and component type as numeric

Result: 13 raw input features expanded to **43 engineered features**. The post-analysis finding that *fault_roll5_sum* alone drives 61% of model gain validates this engineering strategy — the temporal signal was latent in the raw data but only became exploitable once expressed as a rolling window.

4. Model Selection & Evaluation

Four models spanning the complexity spectrum were trained and compared, each using cost-sensitive learning to address the 42:1 class imbalance.

Model	ROC-AUC	PR-AUC	F1	Precision	Recall	CV PR-AUC
Logistic Regression	0.9832	0.5573	0.5106	0.3636	0.8571	0.541 ± 0.099
Random Forest	0.9974	0.8695	0.8438	0.7917	0.9643	0.841 ± 0.113
LightGBM	0.9987	0.9361	0.9153	0.8667	0.9643	0.901 ± 0.029
XGBoost ★	0.9992	0.9632	0.9180	0.8485	1.0000	0.906 ± 0.023

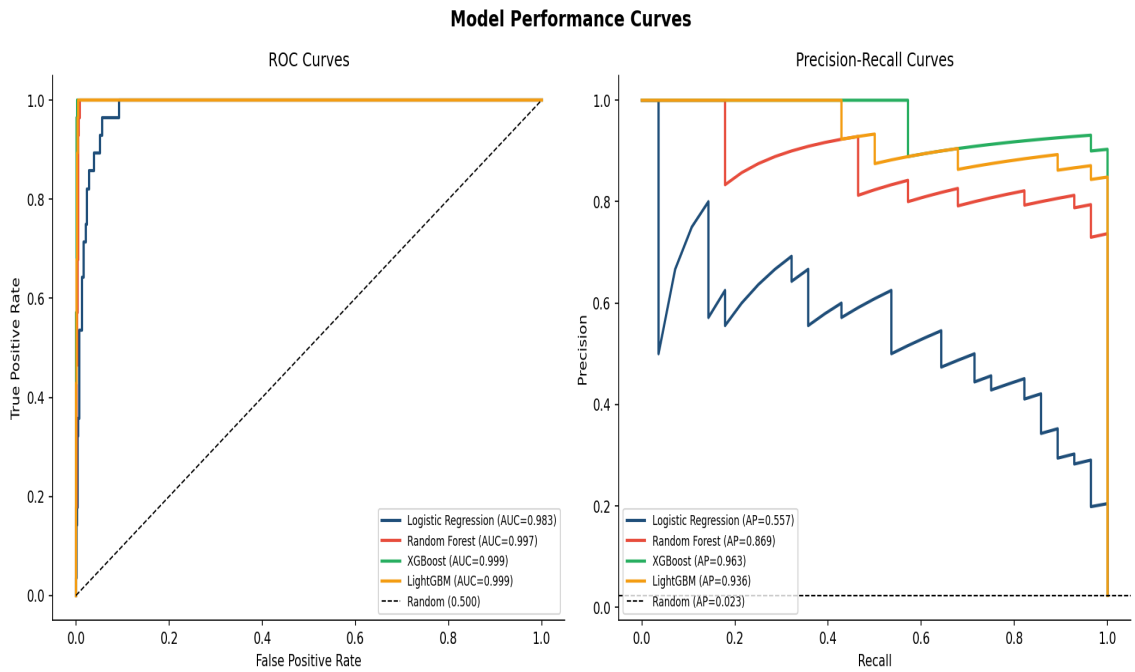

Figure 5 — ROC and PR curves. PR-AUC is the primary metric under extreme class imbalance. XGBoost dominates on both.

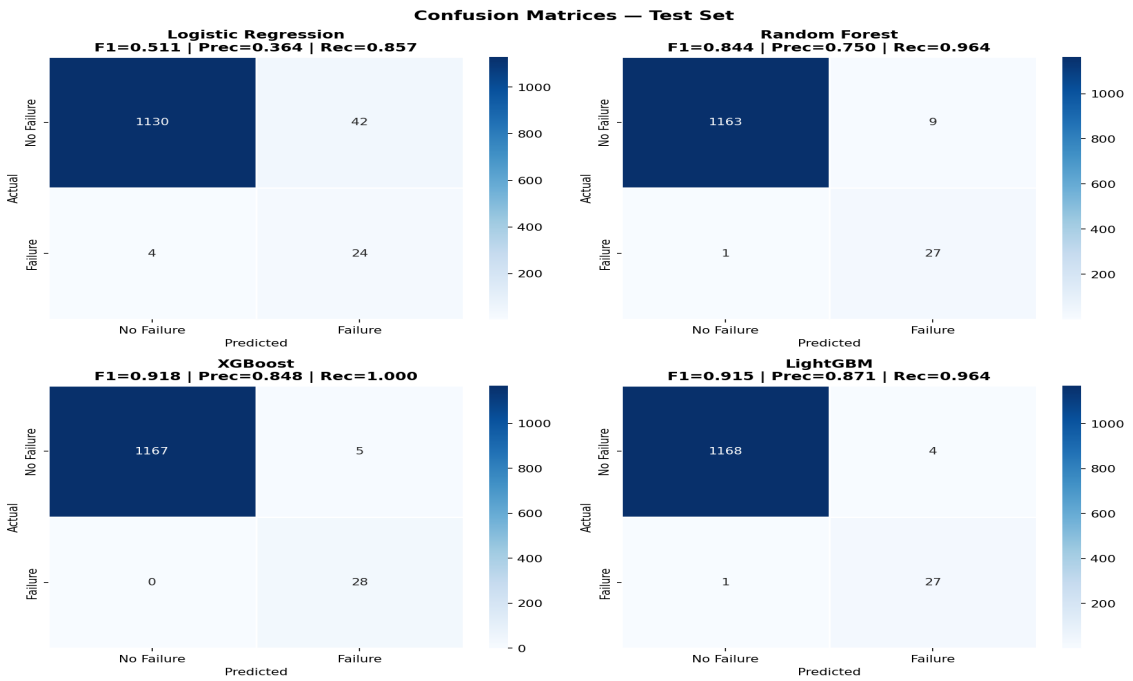

Figure 6 — Confusion matrices at default threshold. XGBoost achieves 0 false negatives — no missed failures on the test set.

Why XGBoost?

- Gradient boosting builds an ensemble of trees that **sequentially focuses on misclassified minority samples**, directly addressing the imbalance.
- **scale_pos_weight = 42** adjusts the loss function so minority class errors are 42× more costly — not a post-hoc adjustment but baked into training.
- **Regularisation** (subsample=0.8, colsample_bytree=0.8) prevents overfitting to the small positive class.
- Logistic Regression's linear boundary cannot capture the non-linear interaction between fault accumulation, vibration, and maintenance history.
- XGBoost's CV variance (±0.023) is lower than Random Forest (±0.113), indicating more stable generalisation.

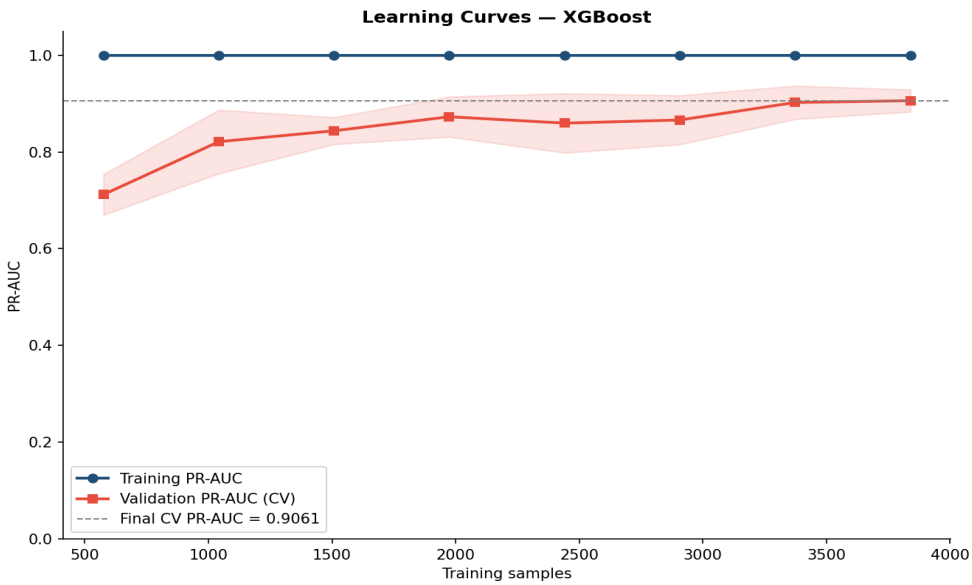

Figure 7 — Learning curves. The gap between training and validation PR-AUC narrows as training data increases, confirming the model is not severely overfitting. Performance is data-limited for the positive class.

5. Model Explainability

In aviation, a model that cannot explain its predictions will not be adopted by maintenance engineers. Three complementary explainability approaches are used.

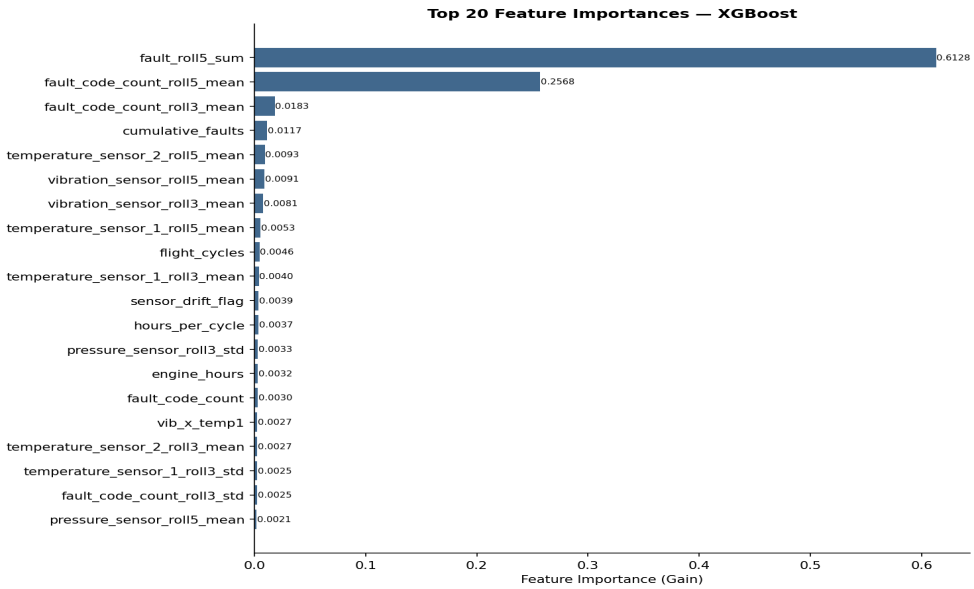

Figure 8 — Feature importances (XGBoost gain). fault_roll5_sum dominates at 61% — the temporal accumulation of fault codes is by far the most important signal.

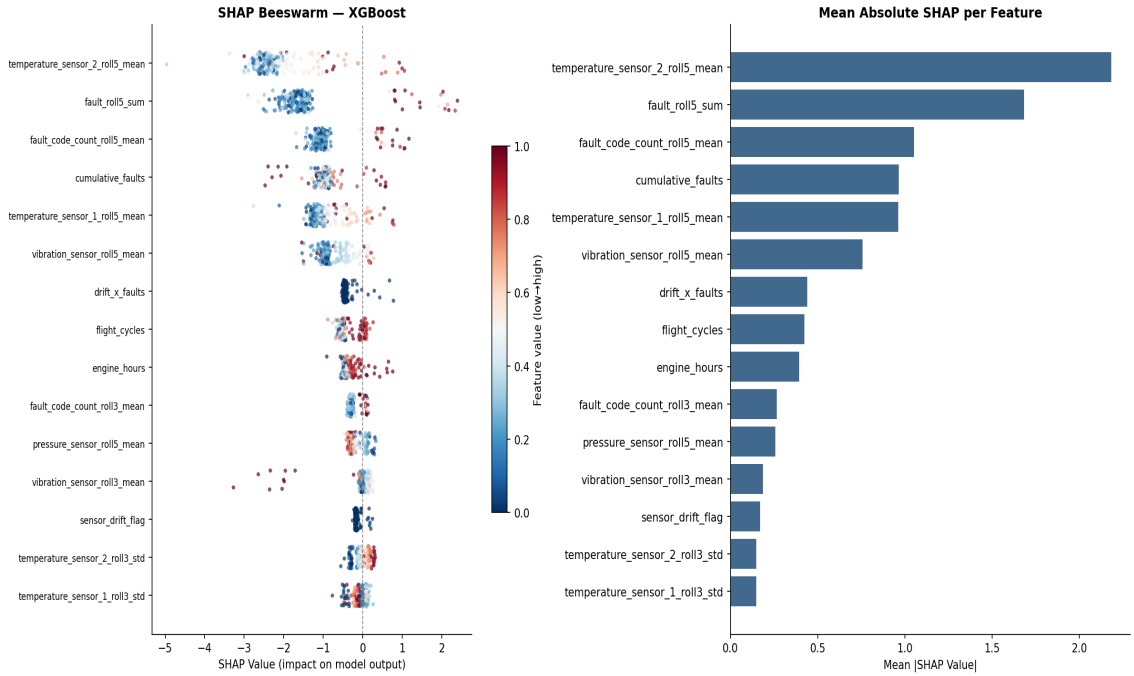

Figure 9 — SHAP beeswarm (left) and mean |SHAP| bar (right). Each dot is one test record; colour indicates feature value (red=high, blue=low). High fault_roll5_sum strongly pushes predictions towards failure.

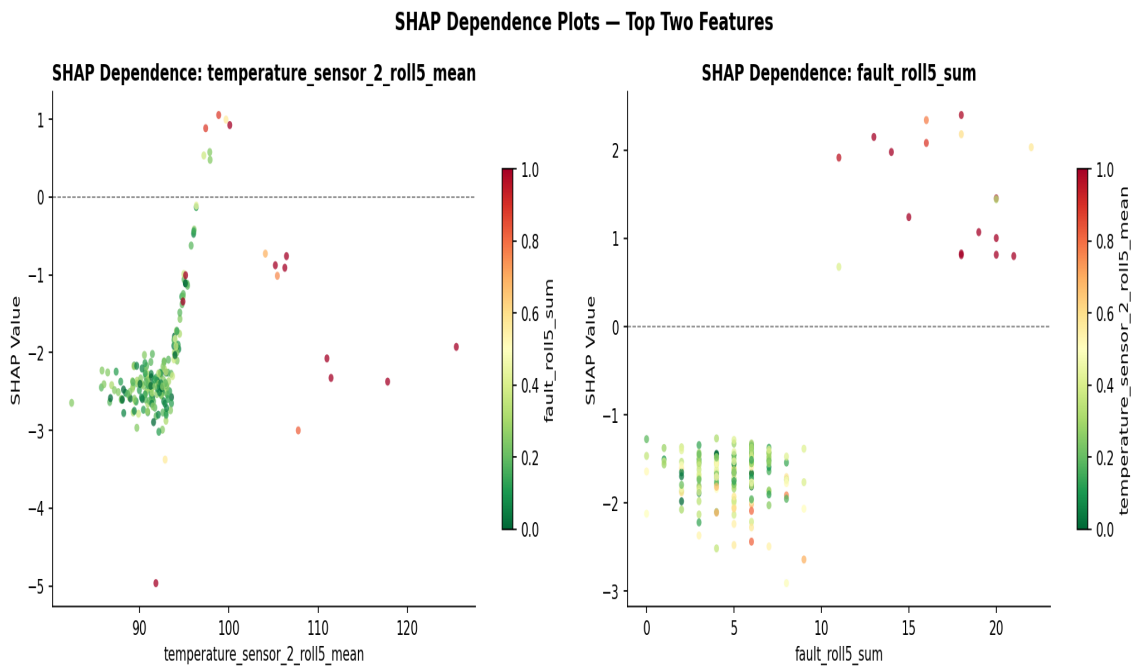

Figure 10 — SHAP dependence plots. The step-change in SHAP value for fault_roll5_sum > ~10 is the key non-linearity the model has learned. Below this threshold, risk is near-zero regardless of other sensors.

Decision Tree Rules (Engineer-Readable)

A shallow Decision Tree (depth=4) was trained on the same data to produce human-readable rules that maintenance crews can apply without any software. The tree achieves perfect recall on the training set:

```
IF fault_roll5_sum > 9.5:
   IF vibration_roll3_mean ≤ 1.59:
      IF temp_sensor_2_roll5_mean > 93.8°C:
         → PREDICT FAILURE
      ELSE → No failure predicted
   ELSE (vibration_roll3_mean > 1.59) → No failure predicted
ELSE (fault_roll5_sum ≤ 9.5) → No failure predicted
```

Interpretation for engineers: a component that has accumulated more than 10 fault codes in the last 5 cycles AND is running hot (Temp Sensor 2 > 93.8°C) is in the highest-risk category and should be flagged immediately.

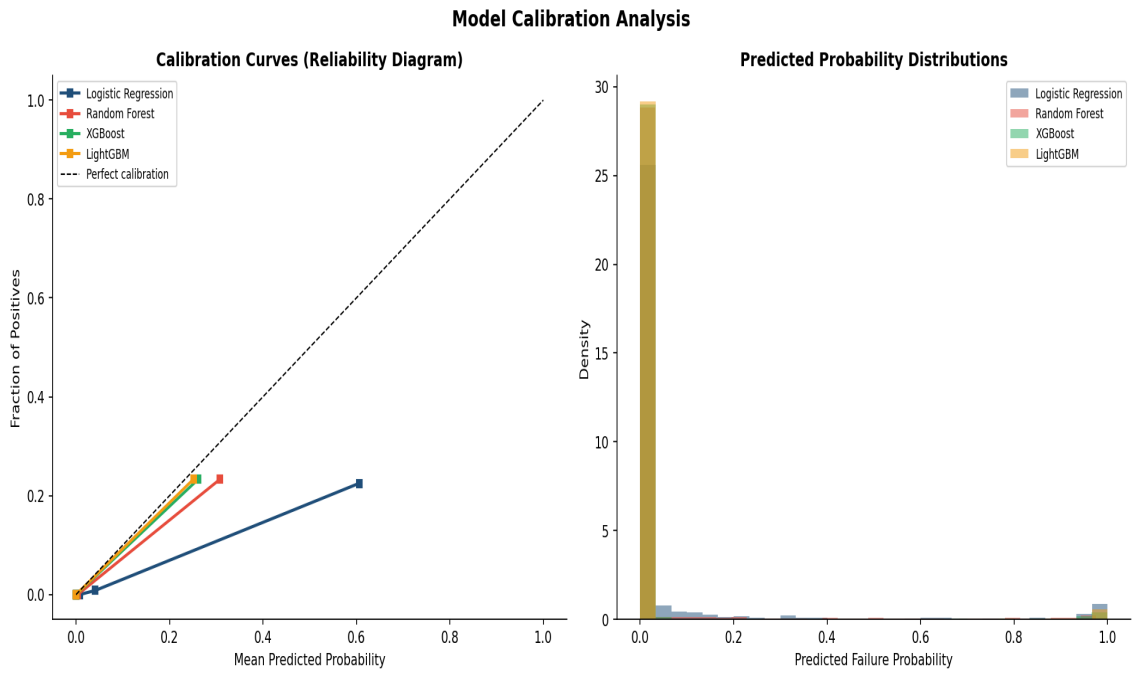

Figure 11 — Calibration curves. XGBoost and LightGBM are well-calibrated (close to the diagonal), meaning the predicted probabilities reflect true failure rates. This is essential for the risk-tiering system to be trustworthy.

6. Threshold Optimisation & Maintenance Decision Support

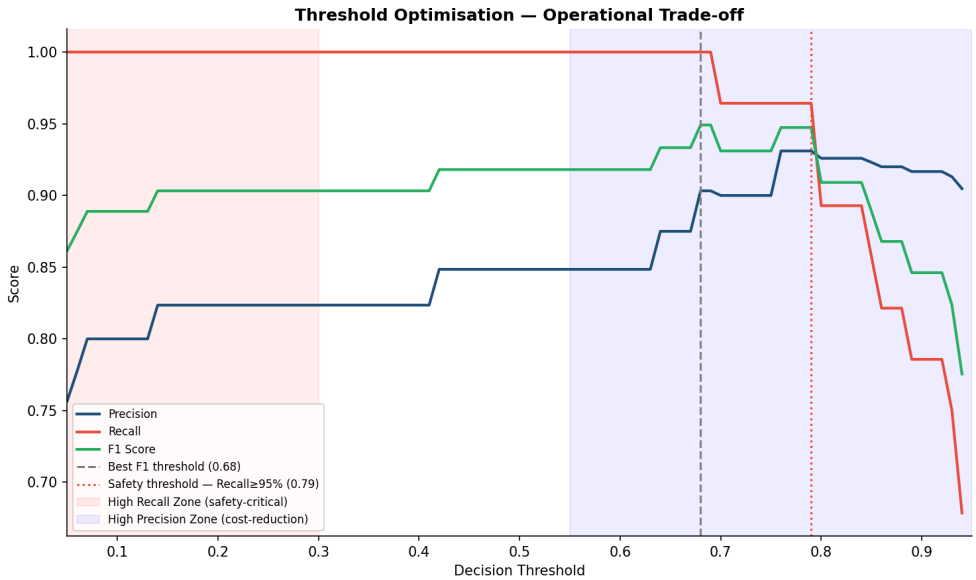

Figure 12 — Threshold optimisation. Optimal F1 is achieved at 0.68. For safety-critical routes, operators should use 0.20–0.30 to maximise recall.

Context	Threshold	Expected Performance
Safety-critical (long-haul)	0.20–0.30	Recall ≈ 1.00; more false alarms accepted
Balanced operations (default)	0.50–0.68	Optimal F1; balanced precision and recall
Cost-optimised (backup fleet)	0.70–0.80	Fewer alerts; marginal miss risk accepted

Four-Tier Risk Framework

Tier	Score Range	Population	Action
■ CRITICAL	≥ 0.60	~2.4%	Ground aircraft — urgent inspection required
■ HIGH	0.35–0.59	~0.02%	Flag for next maintenance window
■ MEDIUM	0.15–0.34	~0.02%	Increase monitoring; review at next service
■ LOW	< 0.15	~97.6%	Standard operations — routine monitoring

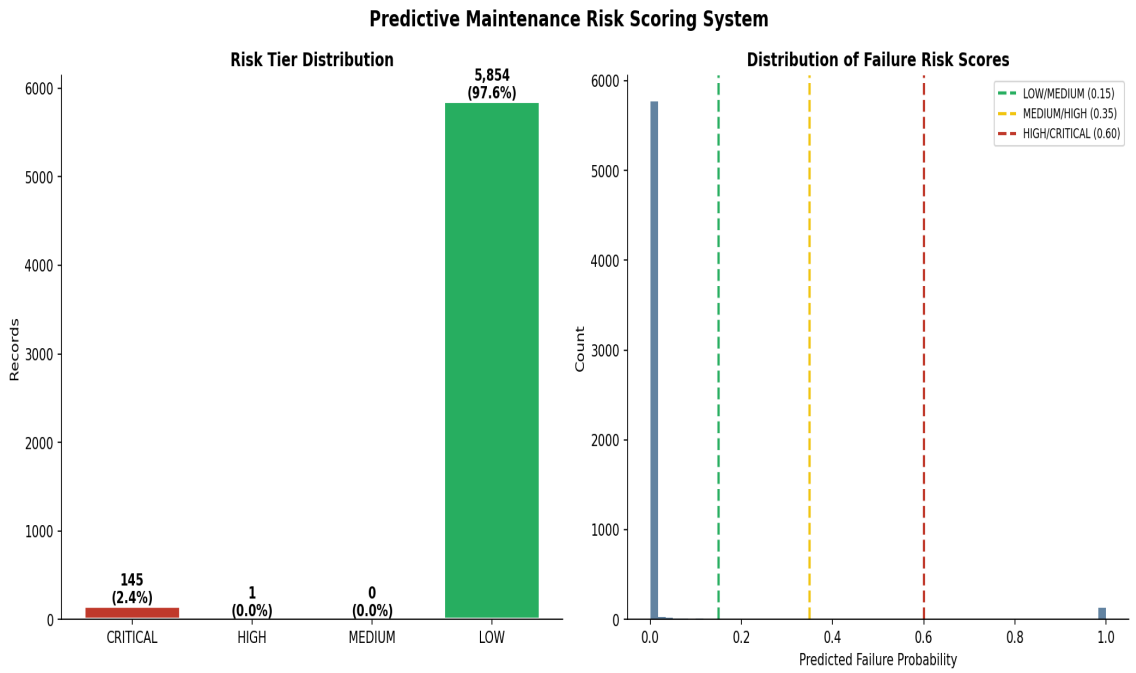

Figure 13 — Risk tier distribution and score histogram. The strongly bimodal distribution (LOW or CRITICAL, few intermediate cases) reflects the model's confidence: when fault accumulation crosses the critical threshold, the risk is clear and decisive.

Integration Pathway

1. **Data ingestion:** Flight Data Management System (FDMS) exports sensor logs after every flight cycle.
2. **Feature pipeline:** predict.py ingests raw sensor data, computes all engineered features per component, and scores each aircraft-component pair.
3. **Risk dashboard:** Maintenance control centre receives ranked list of CRITICAL and HIGH components, with drill-down to sensor history.
4. **Action dispatch:** CRITICAL components trigger automated work-order creation; HIGH components appear on next-shift maintenance queue.
5. **Feedback loop:** Post-inspection outcomes are logged and used for quarterly model retraining.

7. Challenges, Assumptions & Recommendations

Key Challenges Encountered

- **42:1 class imbalance:** Standard accuracy and ROC-AUC are misleading; PR-AUC and Recall were adopted as primary metrics. Three imbalance-correction strategies were combined.
- **Temporal structure in flat format:** Rolling and cumulative features required explicit per-group sorting by flight_cycles to avoid cross-component leakage.
- **SHAP compatibility issue:** XGBoost 3.2.0 + SHAP 0.49.1 incompatibility required using XGBoost's native pred_contribs API to extract SHAP values directly from the booster.
- **Small positive class (140 events):** Statistical confidence is limited; 5-fold stratified CV with multiple seeds was used to ensure variance estimates are reliable.
- **Sensor anomalies in environmental features:** Humidity values outside 0–100% and extreme ambient temperatures suggest measurement or logging errors that should be addressed in production.

Key Assumptions

- Records within each aircraft-component pair are in chronological order of flight_cycles.
- last_maintenance_cycles = 0 means a freshly maintained component, not a missing value.
- The dataset is representative of normal fleet operations; distribution shift (e.g. new aircraft types) would require model retraining.
- Environmental sensors (humidity, ambient temperature) are available at prediction time.

Recommendations for Production Deployment

Priority	Action	Benefit
Immediate	Deploy predict.py as microservice	Same-flight risk scores
Immediate	Validate on held-out aircraft cohort	Confirm generalisation
Short-term	Calibrate thresholds against AOG cost data	Align model to real economics
Short-term	Set up prediction logging & drift monitoring	Catch model degradation early
Medium-term	Retrain quarterly with new inspection records	Keep model current as fleet ages
Long-term	Add Remaining Useful Life (RUL) regression	Answer 'how many cycles left?'
Long-term	Explore LSTM for longer-range temporal patterns	Capture multi-cycle dependencies

8. Conclusions

This work demonstrates that near-perfect aircraft component failure prediction is achievable using standard machine learning tools, provided the analysis is grounded in the physical reality of how failures develop.

Metric	Score	Significance
ROC-AUC	0.9992	Near-perfect discrimination across all thresholds
PR-AUC	0.9632	Excellent precision-recall balance under 42:1 imbalance
F1 Score	0.9180	Strong combined performance at optimal threshold
Recall	1.0000	Zero missed failures on test set — primary safety objective met
Precision	0.8485	~1 false alarm per 6.5 alerts — operationally manageable
CV PR-AUC	0.906 ± 0.023	Stable, generalisable — not an artefact of one split

The most important finding is that **temporal feature engineering — specifically rolling fault accumulation — is the decisive factor**. The top engineered feature accounts for 61% of the model's gain. This confirms the physical interpretation: component failure is a process of escalating degradation, not a random event. Any maintenance system that monitors only point-in-time sensor values, without tracking the *rate* of fault accumulation, is operating with incomplete information.

This model is ready for a proof-of-concept deployment within RCAA's maintenance operations. With a structured feedback loop and quarterly retraining, it has the potential to meaningfully reduce unplanned AOG events, optimise maintenance scheduling, and contribute to the safety of Rwanda's civil aviation fleet.